ELF-DOS Developer's Guide

This guide explains how ELF-DOS programs are built, how the kernel API works, and how to write new programs and commands. For day-to-day use of the finished system, see the *ELF-DOS User's Guide*.

Programs are written in CDP1802 assembly language. This guide assumes you already know that language; it documents ELF-DOS's own conventions on top of it - how a program is loaded and started, what each kernel call does, and the register usage each one expects. Register names `R0` through `RF` refer to the CDP1802's sixteen 16-bit registers (`RA` through `RF` stand for `R10` through `R15`); `D` is the 8-bit accumulator, and `DF` is the 1-bit carry/borrow flag most calls use to report success or failure.

The Toolchain

Programs are built with two tools: `asm02`, the assembler, and `link02`, the linker. Both understand a small set of extensions beyond plain 1802 assembly - a synthetic-opcode mechanism (`.op`, used to define multi-instruction pseudo-ops like `MOV` and `CALL`) and a short-branch relaxation pass (`-r`, which shrinks a long branch to a short one wherever the target is close enough) - that are specific to this toolchain, not standard across every 1802 assembler.

Program Binary Format and Calling Convention

Every program starts with the same six-byte header the kernel itself uses:

```
Offset   Contents
0-2      The letters 'E', 'D', 'F'
3        Program major version number
4        Program minor version number
5        Reserved
6+       Program code -- this is where execution starts
```

This mirrors the kernel's own header exactly (see `KERNEL_HDR_VER` above). Nothing reads a program's own version bytes at load time today - the loader only checks the magic - so a program's version is informational, for whatever the program itself wants to use it for.

Programs are loaded at a fixed address, `PROG_BASE`, defined in `kernel_api.inc`.

At entry:

- `RA` - a pointer to the argument list (`argv`): an array of 16-bit, big-endian pointers, one per argument, each pointing to a plain, null-terminated piece of text. The first argument, `argv[0]`, is the program's own invocation name, matching the C convention `main(argc, argv)`.
- `RC` - the number of arguments (`argc`). Always at least 1.
- `R2` - the stack pointer, already set up and usable.

Every other register, and both `D` and `DF`, start out with an unknown value.

RA and RC are only good until the program's first kernel or BIOS call. After that, either one may have changed, since nothing guarantees a register survives a call unless that call's own documentation says so. Copy whatever you need out of them right away. A minimal skeleton:

```
#include    include/kernel_api.inc

            org     PROG_BASE
            db      'E','D','F'         ; executable signature
            db      1,0,0               ; program version number

start:
            ghi     ra                  ; argv is a table of pointers;
            phi     rf                  ; argv[1] is the first real
            glo     ra                  ; argument (argv[0] is this
            plo     rf                  ; program's own name)
            inc     rf
            inc     rf
            lda     rf
            phi     rd
            ldn     rf
            plo     rd                  ; RD = argv[1]

            ; ... use RD as a pointer to the first argument's text ...

            ldi     0                   ; exit code 0 = success
            rtn
```

To end the program, return with D set to an exit code: 0 for success, anything else is program-defined. A batch script or another program can read this value back through K_GET_ERRORLEVEL (see below).

A program's usable memory - where it can safely put a heap, for example

- is given at a fixed location, LOADER_ARGS, not in a register: word 0 is mem_base, word 1 is mem_top. Read this directly if the program needs to know its own memory bounds, for example to size a heap.

Writing a New Program

A few patterns come up often enough to be worth using rather than reinventing:

- **One required argument**, such as a file name. Check that argc is at least 2, then read argv[1] (shown above).
- **Several independent arguments**, such as deleting more than one file in a single command. Loop over the argument list, handle each one on its own, and if one fails, print an error for it and keep going rather than stopping the whole command - several of the built-in commands (deleting files, copying files, changing attributes) all work this way, and stay quiet when everything succeeds.
- **Wildcards.** A library module provides pattern matching that can be paused and resumed one match at a time (see "Library Modules" below). If a wildcard matches nothing, fall back to using the text exactly as typed, rather than reporting an error.

- **Paths.** Most file and directory calls already understand drive letters and relative paths on their own; hand them the path text directly rather than resolving it yourself first.
- **A heap.** One provided library module is a simple, fast allocator with no way to free a single item, good for "collect everything, use it, then exit" programs. Another is a general-purpose allocator with a real `free`. Both need to be told the program's memory bounds once, by calling their own `init` with the values read from `LOADER_ARGS`.

Kernel API Reference

Every kernel call is reached through a fixed jump table, so a program never depends on the kernel's own internal layout. A call is made the same way as any subroutine call, with arguments in the registers listed below:

```
            ldi     0                   ; mode 0 = read
            call    K_FILE_OPEN
```

New calls are only ever added to the end of the table, never reordered or removed, so a program built against an older kernel keeps working after the kernel is rebuilt.

Unless stated otherwise, a call's `DF` result follows the usual convention: `DF = 0` means success, `DF = 1` means failure. A register not listed under "Returns" should be assumed changed by the call and not relied on afterward.

Working with files

Every open file is represented by a File Control Block (FCB) that the *caller* owns and allocates; there is no kernel-imposed limit on how many files a program can have open at once, only how much memory it is willing to spend. To open a file, a program reserves two blocks of its own memory:

```
            .align  32              ; REQUIRED -- see below
my_fcb:     ds      FCB_LEN         ; 32 bytes -- need not be pre-zeroed
my_iobuf:   ds      FCB_IOBUF_LEN   ; 512 bytes, this FCB's own sector
buffer
```

and passes pointers to both to `K_FILE_OPEN`. Every later call that touches this file - `K_FILE_CLOSE`, `K_FILE_READ`, `K_FILE_WRITE`, `K_FILE_SEEK` - takes the FCB pointer directly; there is no separate small-integer handle to keep track of. The internal layout of an FCB is not documented here and should not be relied on. Treat it as an opaque block the kernel manages on the caller's behalf.

An FCB must not straddle a 256-byte page boundary

The kernel reaches an FCB's fields by adding a field offset to the **low byte** of the FCB pointer alone. That is what makes field access cheap, and it is correct precisely while an FCB's 32 bytes stay inside one page. An FCB split across a page boundary would have its upper fields read from the wrong page.

`K_FILE_OPEN` **rejects** a straddling FCB: it returns `DF` = 1 before touching any of its own state, so the open fails cleanly rather than corrupting memory for the life of the file. There is no distinct error message - it looks like any other failed open. Note what that means in practice: the check fires at *run* time, and only on the builds where the address happens to land badly, so an FCB that is merely *lucky* today can start failing after an unrelated edit shifts the code above it. Align every FCB and it can never happen.

In a flat source file (no `proc`), align the label directly, as in the example above. Inside a `proc`, align the **proc's base** instead and keep the FCB first:

```
            proc    _my_data
            .link   .align 32       ; must be the FIRST thing in the proc
my_fcb:     ds      FCB_LEN         ; offset 0
my_fcb2:    ds      FCB_LEN         ; offset 32 -- also aligned
my_iobuf:   ds      FCB_IOBUF_LEN   ; everything else after
            ...
```

`.link .align` moves the proc's base address, which only aligns a label while nothing has been emitted yet - Link/02 rejects it anywhere else in a proc. Several FCBs can share one aligned proc as long as they are adjacent and come first, so they land at offsets 0, 32, 64 and so on.

An FCB carved out of a heap or a memory reservation at run time must be aligned by the **caller** - round the returned pointer up to a multiple of 32. Nothing checks it for you until `K_FILE_OPEN` refuses it.

`K_FILE_OPEN` Opens a file for reading, or for reading and writing.

- **Args:** `RF` = path, `D` = mode (0 = read, 1 = read/write, creating the file if it doesn't already exist), `RD` = pointer to the caller's `FCB_LEN`-byte FCB, `RA` = pointer to the caller's `FCB_IOBUF_LEN`-byte I/O buffer.
- **Returns:** `DF` = 0/1. `D` is not meaningful on return. The caller already has the FCB pointer it passed in. `DF` = 1 also covers an FCB that straddles a page boundary (see above), which is a caller bug rather than a filesystem condition, and an existing file with `ATTR_RDONLY` set opened in any mode other than 0 (read).

`K_FILE_CLOSE` Closes a file previously opened with `K_FILE_OPEN`.

- **Args:** `RD` = the same FCB pointer passed to `K_FILE_OPEN`.
- **Returns:** `DF` = 0/1.

`K_FILE_READ` / **`K_FILE_WRITE`** Reads or writes bytes at the file's current position, advancing it by the number of bytes actually transferred.

- **Args:** `RD` = FCB pointer, `RF` = data buffer, `RC` = byte count.
- **Returns:** `RC` = bytes actually transferred (read only - a short count usually means end of file), `DF` = 0/1.

`K_FILE_SEEK` Moves a file's current position without reading or writing any data. File positions, offsets, and sizes are tracked as full 32-bit values.

- **Args:** RD = FCB pointer, RC (low byte) = whence (0 = from the start of the file, 1 = relative to the current position, 2 = relative to the end of the file), RA:R9 = signed 32-bit offset (RA = high word, R9 = low word).
- **Returns:** DF = 0 on success, RA:RD = the resulting absolute 32-bit position (RA = high word, RD = low word). DF = 1 if the whence value is invalid, the resulting position would fall outside the file, or an I/O error occurred; the file's position is left unchanged in that case.

K_FILE_DELETE Deletes a file. Refuses to delete a directory or a read-only file.

- **Args:** RF = path.
- **Returns:** DF = 0/1 (not found, is a directory, has ATTR_RDONLY set, or an invalid path component are all errors). D doesn't say which; a program that wants to print "Access denied" can K_STAT the path after a failure and test the bit, as DEL does.

K_FILE_RENAME Renames a file or directory. The new name must stay within the same parent directory - this call cannot move something to a different directory.

- **Args:** RF = old path, RD = new name (a bare name, no path separators).
- **Returns:** DF = 0/1 (the old path not existing, the new name already existing, or either name being `.`/`..` are all errors).

K_FILE_TOUCH Updates a file or directory's last-write date and time to right now. Nothing else about it changes.

- **Args:** RF = path.
- **Returns:** DF = 0/1.

K_FILE_SETATTR Changes a file or directory's attribute byte, by masking it: bits in the "set" mask are turned on, bits in the "clear" mask are turned off, and this is a general-purpose call, not limited to one particular attribute.

- **Args:** RF = path, RC (low byte) = bits to set, RC (high byte) = bits to clear. The new attribute byte is `(old & ~clear) | set`.
- **Returns:** DF = 0/1.

K_STAT Looks up a file or directory without opening it, filling in the same result format K_DIR_READ uses (see "Directories" below). Works on either a file or a directory.

- **Args:** RF = path, RD = pointer to a caller-provided DIRENT_LEN-byte buffer.
- **Returns:** DF = 0 on success (buffer filled), DF = 1 if not found or the path is invalid.

Directories

K_DIR_OPEN Begins a directory listing.

- **Args:** RD = the directory's own starting cluster (0 means the root directory).
- **Returns:** nothing meaningful - a following K_DIR_READ starts from the beginning.

K_DIR_READ Returns the next entry in a directory listing started by K_DIR_OPEN.

- **Args:** RD = pointer to a caller-provided DIRENT_LEN-byte result buffer.

- **Returns:** DF = 0 with the buffer filled in if an entry was available, DF = 1 at the end of the directory. The result buffer has this layout:

Field	Offset	Size	Contents
DIRENT_NAME	0	up to 127 chars	Null-terminated file name.
DIRENT_ATTR	128	1	Attribute byte - see ATTR_DIR/ATTR_HIDDEN below.
DIRENT_CLUST	129	2	First cluster, big-endian.
DIRENT_SIZE	131	4	File size in bytes, big-endian.
DIRENT_WRTTIME	135	2	Last-write time, big-endian, packed: bits 15-11 = hour, 10-5 = minute, 4-0 = seconds/2.
DIRENT_WRTDATE	137	2	Last-write date, big-endian, packed: bits 15-9 = year - 1980, 8-5 = month, 4-0 = day.

  DIRENT_LEN (139) is the total buffer size to declare. ATTR_DIR ($10) is set in DIRENT_ATTR for a subdirectory; ATTR_HIDDEN ($02) is set for a hidden entry; ATTR_RDONLY ($01) for a read-only file.

K_DIR_SAVE_STATE / **K_DIR_RESTORE_STATE** K_DIR_OPEN/K_DIR_READ share one scan position, so only one directory listing can be "in progress" at a time. These two calls let a program pause a listing, do something else (open a file, look something else up), and resume the listing exactly where it left off.

- **K_DIR_SAVE_STATE** - **Args:** RF = pointer to a caller-owned DIR_STATE_LEN-byte buffer. **Returns:** nothing; the buffer is filled in.
- **K_DIR_RESTORE_STATE** - **Args:** RF = pointer to a buffer previously filled by K_DIR_SAVE_STATE. **Returns:** DF = 0 on success (the next K_DIR_READ resumes from the snapshot); DF = 1 if the disk could not be re-read to restore the scan position - treat this the same as any other I/O error.

K_DIR_CREATE Creates a new, empty subdirectory. Single-level only; the parent directory must already exist.

- **Args:** RF = path.
- **Returns:** DF = 0/1 (already exists, an invalid path component, or a full disk are all errors).

K_DIR_REMOVE Removes an empty subdirectory. Refuses a directory that still has entries in it, `.`, `..`, or the root directory itself.

- **Args:** RF = path.
- **Returns:** DF = 0/1.

Paths and drives

K_PATH_RESOLVE Resolves a path - optionally prefixed with a drive letter (`C:` through `F:`) - into a parent directory and a final component, without looking up the final component itself. Every component before the last one must already be a real directory (`.` and `..` work automatically, since they are ordinary directory entries). This is the same resolution logic every path-based file and directory call already uses internally.

Call it directly only when you need the pieces separately, for example to decide whether the final component should name a file or a directory.

- **Args:** `RF` = path (not modified).
- **Returns:** `RD` = the resolved parent directory's cluster. `RF` = pointer to the final path component, null-terminated. This points into kernel-owned scratch memory and stays valid only until the next `K_PATH_RESOLVE` call. Empty if the path ended in a separator or was just `/` (in that case, the cluster in `RD` *is* the target). `RC` (low byte) = the resolved drive index (0-3, 0 = `C:`). `DF` = 0 on success, `DF` = 1 if an intermediate component wasn't found or wasn't a directory, or an explicit drive prefix named a drive with nothing mounted on it.

`K_GETCURDIR` / `K_SETCURDIR` Each drive remembers its own current directory independently, separate from which drive is currently active - the same convention classic MS-DOS uses. Changing a drive's remembered directory does not switch to that drive.

- **`K_GETCURDIR`** - **Args:** none. **Returns:** `RD` = the active drive's current directory cluster (0 = root), `D` = the active drive index (0 to `DRIVE_COUNT`-1).
- **`K_SETCURDIR`** - **Args:** `D` = drive index (0 to `DRIVE_COUNT`-1), `RD` = new current-directory cluster for that drive. **Returns:** nothing.

`K_GETCURDIR` always hands back a usable pair. If the active drive has been unmounted since it was selected, it moves the active drive to the one the shell was loaded from and returns that instead - otherwise the cluster it returned would belong to one drive while the geometry a caller reads it against belonged to another, which showed up as `DIR` listing one drive's contents under another drive's letter.

`K_SETDRIVE` Switches which drive is active.

- **Args:** `D` = drive index (0 to `DRIVE_COUNT`-1) to make active.
- **Returns:** `DF` = 0 on success, `DF` = 1 if that drive has nothing mounted (nothing changes in that case).

`K_GETSHELLDRIVE` Reports which drive the command shell itself was found on at boot - almost always drive 0. This is the drive the system's own commands live on, and the one guaranteed to stay mounted.

- **Args:** none.
- **Returns:** `D` = that drive's index.

`K_DRIVE_INVALIDATE` Drops the cached state belonging to one drive, so its entry in the drive table can be replaced or cleared. Flushes and then discards the FAT cache if that drive is the active one, and forces the next drive switch to reload the table rather than assume it is already current. Used by `MOUNT`, `UMOUNT` and `FORMAT`.

- **Args:** `D` = drive index.
- **Returns:** `DF` = 0 always.
- **Call this *before* changing the drive's table entry, never after.** The flush works out where to write from the *currently active* geometry, so running it against an entry that has already been replaced would write a cached sector to an address computed for the new partition.
- **Writing sectors directly?** Call it before your `K_SECWRITE` calls and again after them. The first call makes sure no pending FAT change lands on top of your writes later; the second makes sure the kernel holds no stale copy of a FAT sector you changed. The FAT sector is the only thing the kernel caches between calls, so nothing else needs this.

> **A drive index is not a letter.** Since drive letters became assignable, index and letter are separate: any of the `DRIVE_COUNT` slots may carry any letter from `A:` to `Z:`. Everything in this API takes an index. Use `lib/drives.asm` to convert either way.

Console input and output

`K_TYPE` Writes one character to the console.

- **Args:** `D` = the character.
- **Returns:** nothing meaningful.

`K_MSG` Writes a null-terminated string, pointed to by a register, to the console.

- **Args:** `RF` = pointer to the string.
- **Returns:** nothing meaningful.

`K_INMSG` Writes a null-terminated string that immediately follows the call instruction itself, rather than being pointed to by a register - convenient for a short literal message:

```
            call    K_INMSG
            db      "Hello.",13,10,0
```

- **Args:** none (the text follows the call in the code itself).
- **Returns:** nothing meaningful; execution continues right after the null byte.

`K_READ` Reads a single character, blocking until one is available. Aware of input redirection on its own: reading from a redirected file returns each byte from that file in turn; reading from a live keyboard blocks for a real keystroke.

- **Args:** none.
- **Returns:** `D` = the character read. Once a redirected file has run out, or input is redirected from the null device, every further call returns `D` = 0 - the same value a genuine null byte in the file would produce, so this call cannot tell the two apart on its own. A program that needs to read whole lines, with a real and unambiguous end-of-file signal, should use `read_line_ex` from the `lineedit.asm` library module instead (see "Library Modules" below).

`K_NBREAD` Reads a single character from the console, giving up after a timeout instead of blocking forever. A direct passthrough to the BIOS's `f_nbread`.

- **Args:** `RC` = the timeout, as a count of BIOS polling loops (0 means 65536). How long one loop takes depends on the BIOS and the clock speed; on EDOS-mbios it is 12 machine cycles, which is 24 microseconds at 4 MHz. The timeout only covers the wait for a character to start arriving.
- **Returns:** `DF` = 0 with the character in `D` and what was left of the count in `RC`, or `DF` = 1 on timeout with `RC` = 0.

On timeout, do not rely on what is left in `RC`.

Like `K_READ`, this follows input redirection. Reading from a redirected file returns each byte at once, leaving `RC` alone, and the end of the file - or input redirected from the null device - is reported as a timeout.

So unlike K_READ, the end of the input can be told apart from a null byte.

f_nbread is a BIOS extension. At boot the kernel checks for a long branch at its vector ($FF84); on a BIOS without one, K_NBREAD becomes a plain blocking read that waits as long as K_READ would and always returns DF = 0. A program then gets no timeout, but still works. To find out, compare the word at IO_NBREAD_TARGET with f_nbread ($FF84): any other value means the blocking read is in use. The check looks at one byte only, so a BIOS that keeps a different routine's vector at $FF84 is not detected.

K_TTY A direct passthrough to the console's own single-character output routine, bypassing any output redirection. Rarely needed; K_TYPE is the ordinary choice.

- **Args:** D = the character.
- **Returns:** nothing meaningful.

Replacing the console with your own

The K_* table is not read-only. Every entry is a three-byte long branch living in RAM, so a program can point one at its own routine and take over that call for the whole system. This is how a custom console - a graphics display, a parallel keyboard - can be installed without changing the kernel.

The interesting entries for a console are K_MSG and K_INMSG. Both render a whole string, so a display that can paint a string faster than one character at a time has something real to gain by taking them over; by default they simply loop over the string calling K_TYPE for each byte.

Each entry is laid out as an opcode byte followed by a two-byte big-endian address:

Entry	Slot	Address bytes to overwrite
K_TYPE	$011E	$011F-$0120
K_MSG	$0121	$0122-$0123
K_INMSG	$0124	$0125-$0126
K_READ	$0151	$0152-$0153
K_NBREAD	$0193	$0194-$0195

Writing the two address bytes is the whole mechanism - leave the opcode byte alone:

```
            mov     rf, K_MSG+1         ; the address field, not the slot
            ldi     high my_fast_msg
            str     rf
            inc     rf
            ldi     low my_fast_msg
            str     rf
```

Save the previous contents first and put them back before exiting, unless the takeover is meant to outlive the program.

K_MSG and K_INMSG need only the one write. Nothing in the kernel ever touches those two entries, so a hook there stays put.

K_TYPE and K_READ need a second write. The kernel rewrites those two itself: when a command redirects its input or output it points them at its own file-writing and file-reading routines, and when the command finishes it puts them back. What it puts back is whatever the words IO_TYPE_TARGET and IO_READ_TARGET contain - the console routines found at boot. So a hook that writes only the table entry is undone by the very next command, redirected or not.

Writing the hook's address to the matching word as well as to the table entry is all it takes. The kernel then restores the hook rather than the boot routine:

```
; take over single-character output
mov     rf, K_TYPE+1        ; the live entry ...
call    store_my_addr
mov     rf, IO_TYPE_TARGET  ; ... and the record the kernel
call    store_my_addr       ;     restores it from
```

Read those two words as "the console output and input routines as they are now", not "as the BIOS supplied them". Replacing the console means replacing what they name. Keep the original contents if the hook is ever to be removed.

K_NBREAD works the same way as K_READ, with IO_NBREAD_TARGET as its word. A replacement console that takes over K_READ should take over K_NBREAD too, or timed reads will still go to the BIOS. Its routine must return DF = 0 with the character in D, or DF = 1 once the count in RC runs out.

A hook must preserve the registers the stock routine preserves. K_INMSG saves and restores RF, RC, R9, RA and RD, and callers do rely on it - MEM, for instance, computes a value in RD, prints a label with K_INMSG, then formats RD. K_MSG preserves RA, and leaves RF pointing at the string's terminating null.

K_INMSG finds its text through R6, which the call itself sets up to point just past the call instruction. Reaching a hook through the table's own long branch preserves that; reaching it through a further nested call would not, because the nested call resets R6. Branch to your routine, never call it, if you chain onward from a hook.

Handling redirection. Output redirection works by pointing K_TYPE at a routine that writes to a file. K_MSG and K_INMSG inherit that for free precisely because they loop through K_TYPE; a hook that paints the screen directly would bypass it, and SOMECOMMAND > FILE would draw on the display instead of writing the file.

To avoid that, a hook can ask whether the console is currently live by comparing K_TYPE's address field against the word at IO_TYPE_TARGET. The test works whether or not the console has been replaced, because a hook updates both together and redirection only ever changes the entry:

- **equal** - output is not redirected; use the fast path.
- **different** - output is going somewhere else; fall back to looping over the string calling K_TYPE, exactly as the stock routine does.

The same comparison works for input, using K_READ against IO_READ_TARGET.

`K_GETDEV` Reports which peripheral devices the BIOS detected at boot (for example, whether a real-time clock is present).

- **Args:** none.
- **Returns:** device flags in `D` - see the BIOS's own documentation for the bit layout, which this call passes through unchanged.

Clock

`K_GETTOD` / `K_SETTOD` Read or set the time-of-day clock, if the hardware has one.

- **Args (`K_SETTOD`):** see the BIOS's own time-of-day format.
- **Returns:** the current time-of-day value, in the same format.

Raw disk access

These two calls bypass the file system entirely and address the disk directly by sector number. **Use them with real care** - a mistake here can corrupt the file system or the disk's own boot sectors. They exist for the rare program (a disk-label editor, an installer) that needs to touch a specific on-disk byte with no ordinary file-system call to reach it.

`K_SECREAD` / `K_SECWRITE` Reads or writes one raw 512-byte sector by its logical block address.

- **Args:** `R7`/`R8` = the 24-bit sector address (`R8` low byte = bits 23-16, `R7` high byte = bits 15-8, `R7` low byte = bits 7-0), `R8` high byte = the block device *unit* number, 0 to 7, `RF` = pointer to a 512-byte buffer (the data to write, or where to put what's read).
- The unit ELF-DOS booted from is the byte at `BOOT_UNIT` (usually 0, but a multi-disk ROM can boot any unit). When working on a drive's own sectors (a volume label, a FAT, a directory), use the unit that drive lives on: a drive can be mounted from another device with `MOUNT`, and writing the boot unit at that drive's addresses damages the boot device instead. The unit is byte `BPBBLK_DEV` of the active drive's parameter block (`BPB_DATA_PTR` in `kernel_api.inc`). A machine whose ROM supports only one device ignores the number entirely.
- **Returns:** `DF` = 0/1. `R7`/`R8` are not preserved across the call.

Memory

`K_HIMEM_RESERVE` / `K_HIMEM_RELEASE` General-purpose reservation of a block of high memory, for anything that needs temporary space beyond its own normal allocation, such as loading a relocatable module, for example. Pure mechanism: no flags, no automatic tracking: the caller is responsible for releasing exactly what it reserved.

`K_HIMEM_RESERVE`

- **Args:** `RC` = bytes to reserve.
- **Returns:** `DF` = 0 with `RD` = the reservation's base address. `DF` = 1 if there isn't enough headroom (nothing changes in that case).

`K_HIMEM_RELEASE`

- **Args:** `RC` = bytes to release - must match a prior successful `K_HIMEM_RESERVE` call exactly.
- **Returns:** nothing.

Miscellaneous

`K_GET_ERRORLEVEL` Reads back the exit code of the last command that ran.

- **Args:** none.
- **Returns:** `D` = the last exit code (0-255), `DF` = 0 always.

Constants Worth Knowing

Name	Value	Meaning
`PROG_BASE`	(see `kernel_api.inc`)	The fixed address every program loads to.
`LOADER_ARGS`	`PROG_BASE - 4`	Word 0 = `mem_base`, word 1 = `mem_top` - the program's usable memory range.
`FCB_LEN`	32	Size of a File Control Block a program must allocate for each open file. Must be 32-aligned - see Working with files.
`FCB_IOBUF_LEN`	512	Size of the I/O buffer that goes with each FCB.
`DIRENT_LEN`	139	Size of the result buffer `K_DIR_READ` and `K_STAT` fill in.
`DIR_STATE_LEN`	9	Size of the snapshot buffer `K_DIR_SAVE_STATE`/`K_DIR_RESTORE_STATE` use.
`IO_TYPE_TARGET`	`PROG_BASE - 114`	Word naming the current console output routine. The kernel restores `K_TYPE` from it after every command, so a console hook must update it too; comparing it against `K_TYPE`'s address field also tells a hook whether output is redirected.
`IO_READ_TARGET`	`PROG_BASE - 112`	The same, for console input and `K_READ`.
`IO_NBREAD_TARGET`	`PROG_BASE - 117`	The same, for timed console input and `K_NBREAD`.
`TERM_ROWS`	`$0191`	Byte: the console terminal's height in rows, 0 if unknown. Kept by the kernel, set by `TERMSIZE`. To lay out output, call `term_size` (`term.asm`), which falls back to `ROWS` and then 24; read the byte directly only where opening the environment file is too slow.
`TERM_COLS`	`$0192`	Byte: the terminal's width in columns, 0 if unknown (assume 80). `read_line_ex` wraps long lines by it. Values over 255 are stored as 255.
`BOOT_UNIT`	`PROG_BASE - 115`	Byte: the block device unit the system booted from, where C:-F: live. Set at boot.

Name	Value	Meaning
DRIVE_COUNT	6	How many drives can be mounted at once. A drive index runs from 0 to DRIVE_COUNT-1 and says nothing about the drive's letter.
MBR_PART_COUNT	4	Primary partitions in an MBR partition table. Deliberately separate from DRIVE_COUNT; a partition number is 1 to 4 however many drives exist.
ATTR_DIR	$10	DIRENT_ATTR bit for a subdirectory.
ATTR_HIDDEN	$02	DIRENT_ATTR bit for a hidden entry.
ATTR_RDONLY	$01	DIRENT_ATTR bit for a read-only file. K_FILE_OPEN refuses modes 1 and 2 and K_FILE_DELETE refuses the file; reading and renaming are allowed. Ignored on a directory.

Library Modules

A handful of library modules are provided alongside the kernel API, ready to link into a program that needs them. None of them are stand-alone programs; they have no header of their own, and are assembled separately and linked into whichever program wants to use them.

Some modules need another one linked alongside them. env.asm and pathstr.asm both call drive_letter_of from drives.asm, so a program using either must link drives.asm too. Leaving it out shows up as an unresolved drive_letter_of at link time.

Module	What it provides
drives.asm	Converting between a drive index and its letter, either way.
env.asm	Reading, setting, and removing environment variables. A whole NAME=VALUE line is limited to ENV_LINE_MAX (128) bytes.
file_glob.asm	Wildcard (*/?) matching that can be paused and resumed one match at a time. glob_init takes a flags byte in D: 0 skips hidden and system entries, as MS-DOS wildcards did; GLOB_HIDDEN (include/file_glob.inc) matches them too. Load it with ldi after setting RF/RD, since a mov clobbers D.
fmt32.asm	Formatting a large (32-bit) number with comma grouping.
heap_bump.asm	A simple, fast memory allocator with no per-item free.
heap_malloc.asm	A general-purpose allocator, with free and coalescing of freed blocks.
icall.asm	Safely calling through an address that is only known while the program is running.
lineedit.asm	read_line_ex: reading a typed line with cursor movement and editing - arrow keys, Home/End, and so on - including lines longer than the screen is wide. RF = buffer, RC.0 = maximum length, RC.1 = the column the input starts at (your

Module	What it provides
	prompt's length; `LE_COL_UNKNOWN` turns wrapping off), `D` = an `LE_MODE_*` value from `include/lineedit.inc`. With `LE_OPT_HIST` ORed into `D`, the Up and Down arrows return to you as `LE_KEY_UP`/`LE_KEY_DOWN`; put another line in the buffer if you like and call `read_line_resume` (`D` = 1 if you changed the buffer, 0 if not) to carry on editing.
`modload.asm`	Loading a relocatable module at whatever address is currently free.
`move.asm`	Renaming a file where possible, falling back to copy-then-delete otherwise.
`pathstr.asm`	Turning a directory's starting cluster back into a full path string.
`term.asm`	`term_size`: the screen size, `RC.1` = rows, `RC.0` = columns (1-255 each). Takes the kernel's `TERM_ROWS`/`TERM_COLS`, else the `ROWS`/`COLUMNS` variables, else 24 by 80, each dimension separately. Needs `env.asm` and `drives.asm`, and may clobber every other register.
`vollabel.asm`	Reading and writing a drive's volume label.
`ymodem.asm`	The YMODEM file-transfer protocol.